CSA17 Artificial Intelligence - Annexure A - Dry Run Evaluation

Answers for Questions 1, 5, 12, 14 and 18

Question 1 - Breadth-First Search (BFS)

Iteration	Queue (before)	Node visited	Queue (after)
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	(empty)

1. BFS traversal order

A, B, C, D, E, F, G

2. Queue contents after each iteration

[B, C] | [C, D, E] | [D, E, F, G] | [E, F, G] | [F, G] | [G] | [ ]

3. Data structure and why level by level

BFS uses a FIFO queue. Nodes are removed from the front and children are added at the back, so all nodes at depth d are dequeued before any node at depth d+1 (their children sit behind them in the queue). This makes the search proceed level by level.

Question 5 - Depth-Limited Search (DLS), limit = 2

Step	Node visited	Depth	Note
1	S	0	Root, not goal. Expand A then B.
2	A	1	Not goal. Expand C then D.
3	C	2	Not goal. At limit, so child H is not generated (cutoff).
4	D	2	Not goal. At limit, so child I is not generated (cutoff). Backtrack to S.
5	B	1	Not goal. Expand E then F.
6	E	2	Not goal. Leaf, dead end.
7	F	2	Not goal. At limit, so child G (depth 3) is not generated (cutoff).

1. Order of nodes visited

S, A, C, D, B, E, F

2. Is goal G found?

No. G is at depth 3 (S -> B -> F -> G), but the depth limit is 2, so the search never goes below depth 2 and returns cutoff. DLS is incomplete when the limit is smaller than the goal depth.

3. Smallest depth limit that finds G

3, since G lies at depth 3.

Question 12 - Local Beam Search (k = 2)

Iter	Current beam	All successors (value)	New beam (best 2)	Continue / stop
1	A(4), B(3)	C(6), D(5), E(9), F(2)	E(9), C(6)	Continue (9 > 4)
2	E(9), C(6)	G(12), H(8), I(7), J(10)	G(12), J(10)	Continue (12 > 9)
3	G(12), J(10)	K(11), L(14), M(9), N(13)	L(14), N(13)	Continue (14 > 12)
4	L(14), N(13)	O(12), P(10), Q(11), R(12)	No improvement (best successor 12 < 14)	Stop
5	-	-	-	Not needed

1. State returned

L (value 14), the best state in the final beam.

2. Why B's branch stays in the beam

Beam search keeps the best k successors pooled from all states in the beam, not the best from each parent. B's child E (9) was higher than every child of A (C = 6, D = 5), so E replaced both A's children except C. The starting value of B does not matter; only the values of the successors do. B's branch then led to L (14), the best state found.

Question 14 - Online Depth-First Search

Maze: top row S, A, B (rock); bottom row C, D, G. Open moves: S-A, S-C, C-D, D-G. Walls block A-B, A-D and B-G. Action order: Right, Down, Left, Up.

Step	Current state	untried[s] (after choosing)	unbacktracked[s]	Action chosen
1	S	{Down}	{ } (start)	Right -> A
2	A	{ } (only legal action Left chosen)	{S}	Left -> S (dead end, backtrack)
3	S (revisited)	{ } (Down chosen)	{ }	Down -> C
4	C	{Up}	{S}	Right -> D
5	D	{Left}	{C}	Right -> G
6	G	-	-	Goal reached, stop
7	-	-	-	Not needed

1. Path actually travelled

S -> A -> S -> C -> D -> G (5 moves)

2. Why the agent returns to S after A

A is a dead end: Right is the rock B, Down is a wall, and Up is the boundary, so the only legal action at A is Left. With no other untried action, the agent backtracks to S, where the untried action Down is still available.

3. Competitive ratio

Actual path cost = 5. Optimal path S -> C -> D -> G costs 3. Competitive ratio = 5 / 3 = 1.67 (approximately).

Question 18 - Min-Conflicts (4-Queens)

State = rows of Q1..Q4. Start state (1, 1, 2, 3).

Step	State	Attacking pairs	Queen chosen	Conflicts in each row (rows 1, 2, 3, 4)	Move
1	(1,1,2,3)	4: Q1Q2, Q2Q3, Q2Q4, Q3Q4. Conflicts per queen: Q1=1, Q2=3, Q3=2, Q4=2	Q2 (most conflicts, 3)	Q2: row1 = 3, row2 = 2, row3 = 2, row4 = 0	Q2: row 1 -> row 4
2	(1,4,2,3)	1: Q3Q4. Q3=1, Q4=1	Q3 (leftmost tie)	Q3: row1 = 1, row2 = 1 (current), row3 = 3, row4 = 2	Q3: row 2 -> row 1
3	(1,4,1,3)	1: Q1Q3. Q1=1, Q3=1	Q1 (leftmost tie)	Q1: row1 = 1 (current), row2 = 0, row3 = 3, row4 = 1	Q1: row 1 -> row 2
4	(2,4,1,3)	0	-	-	No queen attacked, stop
5	-	-	-	-	Not needed

1. Final solution

(2, 4, 1, 3): Q1 in row 2, Q2 in row 4, Q3 in row 1, Q4 in row 3. No two queens share a row, column or diagonal.

2. Why min-conflicts solves even million-queens quickly

It is a local repair method: it starts from a full assignment and only changes one queen at a time to the row with the fewest conflicts, so it never builds or backtracks over partial solutions. Conflict counts can be updated incrementally with cheap row and diagonal counters. Also, n-queens has very many solutions that are densely spread across the state space, so the number of steps needed stays almost constant (about 50) even as n grows to millions.